OPCODA: SÍNTESE GRANULAR ORIENTADA PELA ANÁLISE ESTRUTURAL E ESTATÍSTICA DE BINÁRIOS PE

Mateus Costa de Oliveira Santos

Rafael Augusto Garcez Elisei

Prof. Me. Warner Brezolin (orientador)

RESUMO

Este trabalho apresenta o Opcoda, um instrumento que emprega arquivos executáveis do formato Portable Executable como matéria-prima e, simultaneamente, como mecanismo de controle da síntese granular. A proposta parte da identificação de uma lacuna entre dois tratamentos correntes. O databending converte arquivos digitais em áudio por importação de dados brutos e, com isso, reduz o objeto de origem a uma sequência linear de valores, da qual se descarta a organização interna; os sintetizadores granulares convencionais operam sobre amostras previamente preparadas, nas quais a estrutura do arquivo que as originou não é recuperável nem utilizada. Nenhuma das duas abordagens estabelece uma relação verificável entre as propriedades estruturais e estatísticas do arquivo e os parâmetros da síntese. O Opcoda reúne as duas etapas em um instrumento, no qual cada região declarada na tabela de seções do PE constitui fonte granular independente e a entropia de Shannon, calculada sobre janelas de 2048 bytes, é convertida em variável de controle da distribuição dos grãos. A viabilidade dessa integração foi verificada sob as restrições de tempo real do processamento de áudio: a leitura estática e verificada do arquivo, a conversão dos bytes e o condicionamento de sinal ocorrem fora do laço de processamento, e a comunicação com o motor de síntese se dá por fila circular sem trava com ponteiro cru e ordenamento acquire-release. O protótipo foi implementado em C++20 sobre a biblioteca JUCE 8, com núcleo independente de framework de áudio, e gera aplicativo autônomo e plugin no formato VST3 para Windows x64. A suíte de verificação automatizada reúne 196 casos de teste unitário e de integração, mais 8 casos do guard de alocação, que transformam a restrição de tempo real em critério verificável por build. O ensaio espectral confirmou atenuação da componente de corrente contínua acima do critério de 40 dB e energia abaixo de 20 Hz abaixo do critério de -60 dBFS, e os casos de borda do parser foram rejeitados com erro tipado, sem falha de segmentação. A validação dinâmica de latência e o ensaio de aceite com o usuário permanecem bloqueados por ausência de estação de trabalho de áudio licenciada na máquina de desenvolvimento, lacuna declarada e não omitida. O trabalho contribui ao converter propriedades estruturais e estatísticas de um artefato binário em parâmetros de controle de um instrumento musical, com a disciplina de tempo real como requisito de viabilidade e não como contribuição equivalente.

Palavras-chave: Síntese granular. Databending. Formato Portable Executable. Entropia de Shannon. Computação musical. Processamento digital de sinais.

ABSTRACT

This work presents Opcoda, an instrument that uses Portable Executable files both as source material and as a control mechanism for granular synthesis. The proposal addresses a gap between two current practices. Databending converts digital files into audio by raw data import, which reduces the source object to a linear sequence of values and discards its internal organization, while conventional granular synthesizers operate on previously prepared audio samples, in which the structure of the originating file is neither recoverable nor used. Neither approach establishes a verifiable relation between the structural and statistical properties of the file and the parameters of the synthesis. Opcoda integrates both stages into a single instrument, in which each region declared in the PE section table constitutes an independent granular source and the Shannon entropy computed over 2048-byte windows is converted into a control variable for grain distribution. The feasibility of this integration was verified under the real-time constraints of audio processing: static and verified file reading, byte conversion, and signal conditioning take place outside the processing callback, and communication with the synthesis engine occurs through a lock-free ring queue carrying a raw pointer with acquire-release ordering. The prototype was implemented in C++20 on top of the JUCE 8 library, with a core independent of any audio framework, and produces a standalone application and a VST3 plug-in for Windows x64. The automated verification suite comprises 196 unit and integration test cases, plus 8 allocation-guard cases that turn the real-time constraint into a build-verifiable criterion. The spectral trial confirmed direct current attenuation above the 40 dB criterion and energy below 20 Hz under the -60 dBFS criterion, and the parser boundary cases were rejected with typed errors and no segmentation fault. Dynamic latency validation and the user acceptance trial remain blocked by the absence of a licensed digital audio workstation on the development machine, a gap that is declared rather than omitted. This work contributes a method for converting structural and statistical properties of a binary artifact into control parameters of a musical instrument, with real-time discipline as a feasibility requirement rather than as an equivalent contribution.

Keywords: *Granular synthesis. Databending. Portable Executable format. Shannon entropy. Computer music. Digital signal processing.*

1 INTRODUÇÃO

1.1 Contexto e motivação

Databending é a prática de usar arquivos não sonoros como fonte musical, tratando seu conteúdo bruto como amostras de áudio. A prática ganhou forma com o glitch e o uso de discos riscados, fitas e discos flexíveis no sample-and-hold de samplers analógicos, e hoje encontra implementação direta em ferramentas de edição: o Audacity, por exemplo, importa qualquer arquivo pelo diálogo de dados brutos, onde o usuário declara codificação, ordem de bytes, canais, deslocamento inicial, percentual do arquivo e taxa de amostragem (AUDACITY TEAM, 2026).

Essa importação resolve o problema de transformar arquivo em sinal, e não o problema de transformar arquivo em instrumento. O procedimento é offline, não tem polyphonia, não responde a notas e não roda dentro do caminho de áudio de uma estação de trabalho. Para quem quer tocar, o resultado serve como material bruto para edição posterior, não como fonte de performance.

Quando o arquivo não sonoro é um executável, o problema se agrava por quatro razões que aparecem de forma consistente. Primeira, o deslocamento de corrente contínua: bytes de programa não têm média zero, opcodes comuns concentram valores em faixas específicas, e o sinal resultante fica deslocado do zero, o que reduz o volume útil e produz estalos. Segunda, transientes espúrios: cabeçalhos DOS, COFF e tabelas de seção, bem como trechos de preenchimento com zeros, criam saltos bruscos de amplitude que soam como cliques sem relação musical. Terceira, aliasing: ler byte a byte como amostra cria variações muito rápidas, acima do limite de Nyquist, que se dobram para dentro do espectro audível de forma descontrolada quando se mexe na afinação ou na posição de leitura. Quarta, risco de instabilidade: um leitor que confia em campos do cabeçalho, como o tamanho declarado dos dados brutos de uma seção ou o número de seções, pode ler fora dos limites do arquivo quando o binário está corrompido ou forjado, e no pior caso derruba a estação de trabalho inteira com falha de segmentação.

Estas quatro razões têm naturezas diferentes. As três primeiras são problemas de qualidade de sinal e têm solução conhecida em engenharia de áudio: janelamento, filtragem e limitação. A quarta é um problema de segurança e de robustez, e é a que determina a arquitetura. Um instrumento que roda dentro do processo do host não pode simplesmente delegar a um leitor de

arquivo a tarefa de extrair material de um artefato arbitrário, porque uma falha ali é uma falha do host.

1.2 Problema de pesquisa

A pergunta que orienta este trabalho é a seguinte: como construir um instrumento em tempo real que leia binários Portable Executable arbitrários com segurança e os converta em material sonoro controlável, por síntese granular guiada por entropia, sem travar ou atrasar o host de áudio?

A hipótese correspondente é que um pipeline com parser isolado, fila sem trava e motor granular com janelamento e filtro de bloqueio de corrente contínua reduz o deslocamento de corrente contínua em pelo menos 40 dB em relação à leitura bruta, mantém o processamento por bloco abaixo de 50% do tempo disponível com oito vozes e registra zero travamentos sob entrada malformada. A hipótese nula é que o tratamento granular e estatístico não produz diferença relevante em relação à leitura bruta direta.

A hipótese é deliberadamente mensurável. Cada componente dela corresponde a um ensaio com critério de aceite escrito antes da implementação: a atenuação é medida por análise espectral da saída, a fração de tempo é medida por latência de bloco e a robustez é medida por execução repetida sobre entradas inválidas. A escolha por números verificáveis, em vez de afirmações qualitativas sobre qualidade, é o que distingue um trabalho de engenharia de uma demonstração.

1.3 Objetivos

O objetivo geral é entregar, até 3 de novembro de 2026, o Opcoda como aplicativo autônomo e plugin no formato VST3 para Windows x64, capaz de abrir ou receber por arrasto arquivos executáveis, com parâmetros granulares controláveis por MIDI, redução da componente de corrente contínua de ao menos 40 dB e reprodução sem falhas de tempo real. O formato CLAP e a monografia completa seguem após essa entrega.

Os objetivos específicos se distribuem em três áreas.

Na área de engenharia de software estão o parser do formato PE, próprio, que valida assinatura MZ, assinatura PE, número de seções e limites de cada seção, e converte bytes

em ponto flutuante normalizado; a ingestão somente leitura, que aceita abertura por diálogo e arrasto do sistema operacional, faz cópia defensiva e devolve códigos de erro claros para arquivos truncados, sem propagar exceção através da interface binária do plugin; e a fila sem trava entre interface e processamento de sinais, com troca de amostra por duplo tampão.

Na área de processamento de sinais estão o motor granular polifônico de oito vozes, com tamanho de grão de 1 a 100 ms, densidade, posição, dispersão, afinação e volume, e janelas Hann e gaussiana; o cálculo de entropia de Shannon por janela de 2048 bytes e por seção, usado para controlar a dispersão e a aleatoriedade dos grãos; e a cadeia de condicionamento, com centralização da corrente contínua, filtro de primeira ordem com coeficiente de 0,9983 e limitador de saída com teto de -1 dBFS.

Na área de interface estão a interface de três faixas com mapa de seções do PE e grelha navegável de bytes, com controles automatizáveis na estação de trabalho; o isolamento entre interface e processamento, sem acesso direto ao estado do áudio, verificado por guard de alocação; e o mapeamento MIDI nativo, com notas, pedal de sustentação e dois controladores de mudança de controle, nos parâmetros granulares, tanto no aplicativo autônomo quanto no plugin.

1.4 Delimitação do escopo

O escopo é delimitado de forma deliberada, e o que fica de fora é parte do argumento.

O Opcoda não executa o binário. Não há análise dinâmica, não há depuração, não há emulação de instruções e não há interpretação semântica do código de máquina. O arquivo é tratado exclusivamente como uma sequência de bytes para leitura, e a hipótese acústica é de que a estrutura estatística dessa sequência, e não o significado das instruções, é o que produz a textura sonora. Essa escolha também é a razão pela qual o trabalho não exige permissão especial para além de leitura de arquivo: um artefato que o usuário poderia executar de outro modo é tratado aqui com o mesmo cuidado com que se trata uma imagem corrompida.

Também estão fora do escopo a análise de malware, o desmonte de código para inspeção, o reconhecimento de assinaturas de empacotadores como critério de correção, a modulação por rede, a reprodução em Linux ou macOS, e a montagem de mais de oito vozes simultâneas. O formato CLAP está previsto no projeto, mas a entrega do MVP é em VST3.

1.5 Contribuições

A contribuição principal do trabalho é a combinação de três elementos que, em nosso levantamento, não aparecem juntos em nenhum sistema existente. O primeiro é um parser do formato PE com verificação de limites dentro de um instrumento, e não em uma ferramenta de análise. O segundo é a entropia de Shannon em janelas usada como sinal de controle de síntese, e não apenas como métrica de diagnóstico forense. O terceiro é a disciplina de tempo real aplicada a um instrumento cuja fonte de áudio é um arquivo arbitrário, o que transforma a ingestão de dados no ponto mais delicado do sistema em vez de um detalhe de implementação.

Como consequência secundária, o trabalho produz evidência sobre o custo real de manter o núcleo de um plugin de áudio livre de framework. A separação entre núcleo e adaptador, adotada aqui por razones de teste e de sanitização, é uma decisão de engenharia com efeitos mensuráveis sobre asuíte de verificação.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 O formato Portable Executable e a leitura estática de arquivos não confiáveis

O formato Portable Executable é o formato de arquivos executáveis, bibliotecas dinâmicas e drivers do sistema Windows nas arquiteturas x86 e x64, derivado do Common Object File Format. Sua especificação é mantida pela Microsoft e descreve a organização binária que o carregador do sistema operacional utiliza para mapear o arquivo em memória (MICROSOFT, 2024).

A anatomia do formato foi documentada por Pietrek em dois textos de referência que permanecem a leitura inicial para quem nunca abriu um executável por dentro: o primeiro descreve cada campo dos cabeçalhos e sua função no carregamento, e o segundo aprofunda os casos que escapam da descrição simplificada (PIETREK, 1994; PIETREK, 2002). A sequência lógica do arquivo é o cabeçalho DOS, mantido por compatibilidade e com o campo de deslocamento que aponta para o cabeçalho PE; a assinatura PE; o cabeçalho COFF, com número de seções e características da imagem; o cabeçalho opcional, com ponto de entrada, endereço-base, tamanhos de pilha e heap e diretórios de dados; e a tabela de seções, seguida do corpo das seções.

Cada entrada da tabela de seções declara nome, tamanho virtual, endereço virtual, tamanho dos dados brutos em disco, ponteiro para esses dados e flags de permissão que distinguem código, leitura e escrita. Essa dualidade entre tamanho em disco e tamanho em memória é central para a leitura segura, porque os dois valores podem divergir tanto de forma legítima, como em dados não inicializados, quanto por corrupção.

As seções convencionais possuem perfis distintos de conteúdo. A seção de código concentra código de máquina executável. As seções de dados e de dados somente leitura concentram dados inicializados e constantes. A seção de recursos concentra ícones, imagens, diálogos e cadeias de caracteres. Sikorski e Honig usam exatamente essa segmentação como ponto de partida da análise estática de binários, extraindo seções para inspeção sem executar o artefato (SIKORSKI MICHAEL; HONIG, 2012). No Opcoda, essa segmentação fundamenta a hipótese acústica do instrumento: cada seção constitui uma região de textura estatística própria, sonificada de forma parametrizável.

A análise estática, por definição, extrai informação do arquivo sem executá-lo. Para arquivos potencialmente não confiáveis, três princípios se aplicam. O primeiro é o princípio do menor privilégio: o arquivo é aberto exclusivamente para leitura e nenhuma página de memória recebe permissão de execução, de modo que a execução acidental é eliminada por construção. O segundo é a verificação de limites antes de cada leitura indexada por campos do cabeçalho, o que previne leituras fora dos limites e estouros de buffer. O terceiro é a rejeição tipada: valores fora de faixas plausíveis produzem erro identificado, sem propagação de exceção através da interface binária do plugin. Esses procedimentos materializam, no domínio de análise de binários, as práticas de software seguro para sistemas com restrições temporais descritas por Kleidermacher e Kleidermacher (KLEIDERMACHER DOMINIK; KLEIDERMACHER, 2012).

Um ponto merece destaque porque costuma ser tratado como detalhe de implementação e decide a sobrevivência do host. O tamanho declarado dos dados brutos de uma seção não é confiável. Um arquivo pode declarar um tamanho maior que o conteúdo efetivamente presente, e essa é uma situação que ocorre em arquivos legítimos com padding de linkador e em arquivos deliberadamente forjados. A única postura segura é tratar o valor declarado como um teto e recortá-lo para o que existe, com rejeição explícita quando o declarado excede o disponível de forma incompatível com a tolerância a padding.

2.2 Teoria da informação e entropia de dados arbitrários

Shannon definiu a entropia de uma fonte discreta como a quantidade média de informação por símbolo, dada pela expressão seguinte, em que $p(x_i)$ é a probabilidade do símbolo x_i no alfabeto da fonte (SHANNON, 1948).

$$H = -\sum_{i} p(x_i) \cdot \log_2 p(x_i) \tag{1}$$

Aplicada a fluxos de bytes, o alfabeto contém 256 símbolos, com valores de 0 a 255, e a entropia máxima é 8 bits por byte, atingida quando todos os valores são equiprováveis.

O cálculo sobre o arquivo inteiro produz um único número, e um único número é insuficiente para navegação paramétrica. A técnica adotada é a janela deslizante de tamanho fixo, neste trabalho de 2048 bytes: estima-se a distribuição de frequências dentro de cada janela e computa-se a entropia local, o que produz uma curva ao longo do arquivo.

A correlação entre entropia e natureza do conteúdo é direta e conhecida da análise forense. Regiões com repetição massiva de bytes, como preenchimento com zeros ou áreas não inicializadas, apresentam entropia próxima de zero. Código compilado padrão apresenta entropia intermediária, refletindo a distribuição enviesada de opcodes e operandos. Blocos comprimidos, empacotados ou cifrados apresentam entropia próxima de 8 bits por byte, comportamento análogo ao ruído estocástico de banda larga. Lyda e Hamrock demonstraram que esse perfil entrópico distingue classes de conteúdo em binários, porque regiões distintas do arquivo exibem assinaturas estatísticas distintas (LYDA RICHARD A.; HAMROCK, 2007).

No Opcoda, a curva de entropia opera como fonte de modulação paramétrica contínua. A entropia local pondera a dispersão temporal dos grãos, sua aleatoriedade de posição e a densidade da nuvem. Regiões de alta entropia geram texturas instáveis e ruidosas; regiões de baixa entropia geram texturas estáveis e periódicas. A justificativa teórica é que a entropia resume, em um escalar por janela, a mesma distinção entre ordem e ruído que a síntese granular manipula no domínio temporal. Uma seção de recursos com muito zeros e uma seção comprimida com bytes aparentemente aleatórios soam diferentes por essa razão, e o instrumento expõe essa diferença como controle de timbre em vez de tratá-la como detalhe de implementação.

A medição por janela impõe um cuidado que a análise forense não precisa tomar: a entropia de uma seção não é a entropia do material que começa no deslocamento da seção. O fim da seção é o deslocamento mais o tamanho, e esse tamanho é o mínimo entre o declarado

e o disponível, de modo que a diferença entre as duas leituras cresce com a distância entre o deslocamento e o fim do arquivo. Uma seção de 256 bytes cheios de zero seguida de 256 bytes de dados uniformes mede aproximadamente 4,98 bits por byte quando medida a partir do deslocamento, e zero quando medida com comprimento explícito.

2.3 Processamento digital de sinais e síntese granular

Roads formalizou a síntese granular a partir do conceito de microsom: o som como agregado de microfragmentos de 1 a 100 ms, cada grão com duração, envelope, frequência, amplitude e posição próprios, organizados em estruturas meso-temporais como nuvens e fluxos (ROADS, 2001). Os parâmetros canônicos são tamanho de grão, densidade de disparo, dispersão temporal, afinação e panorama. Essa teoria sustenta o motor do Opcoda, cujos grãos variam de 1 a 100 ms no MVP.

O problema do material bruto não senoidal precisa ser nomeado com precisão. Interpretar bytes executáveis como amostras PCM produz um sinal sem as garantias do áudio convencional: média diferente de zero, descontinuidades abruptas nos limites de cabeçalhos e regiões de preenchimento, e variações amostra a amostra acima do limite de Nyquist, que se dobram para o espectro audível como aliasing (SMITH, 2007; ZÖLZER, 2011). O janelamento granular e a filtragem de condicionamento existem precisamente para converter esse material bruto em sinal musicalmente utilizável.

Multiplicar cada grão por uma função de janela força amplitude nula nas bordas, o que elimina cliques por descontinuidade de fase na sobreposição. As formulações adotadas, para janela de N amostras com n de 0 a $N-1$, são as três seguintes (SMITH, 2007).

$$w_{Hann}(n) = 0{,}5 \cdot \left(1 - \cos\left(\frac{2\pi n}{N-1}\right)\right) \tag{2}$$

$$w_{Hamming}(n) = 0{,}54 - 0{,}46 \cdot \cos\left(\frac{2\pi n}{N-1}\right) \tag{3}$$

$$w_{Blackman}(n) = 0{,}42 - 0{,}5 \cdot \cos\left(\frac{2\pi n}{N-1}\right) + 0{,}08 \cdot \cos\left(\frac{4\pi n}{N-1}\right) \tag{4}$$

O instrumento adota Hann e gaussiana no MVP. A escolha se fundamenta no compromisso entre suavidade temporal e seletividade espectral: janelas suaves reduzem espalhamento

espectral dos transientes ao custo de resolução temporal, efeito desejável sobre material de databending.

O deslocamento de corrente contínua é removido por filtro de resposta ao impulso infinita passa-alta de primeira ordem, com a seguinte equação de recorrência, em que o coeficiente R posiciona o polo próximo ao círculo unitário (SMITH, 2007).

$$y[n] = x[n] - x[n-1] + R \cdot y[n-1] \tag{5}$$

O coeficiente é dado pela expressão a seguir, e a substituição do valor da frequência de corte mostra por que o número importa mais do que parece.

$$R = \exp\left(-\frac{2\pi f_c}{f_s}\right) \tag{6}$$

Com $R = 0{,}9983$ e $f_s = 44{,}1$ kHz, a frequência de corte é de aproximadamente 12 Hz, dentro da faixa de 10 a 15 Hz que preserva o conteúdo audível. Com $R = 0{,}995$, o mesmo cálculo resulta em 35 Hz, que corta parte do grave que o instrumento deveria preservar. A divergência entre o coeficiente citado no texto e o coeficiente que produz a frequência de corte prometida foi encontrada durante a implementação e está registrada no capítulo de desenvolvimento, porque o critério do ensaio espectral mede a faixa de frequência e não o coeficiente.

A coloração adicional utiliza biquad ressonante na forma direta, com a seguinte equação de recorrência e coeficientes calculados por frequência, fator de qualidade e ganho segundo as fórmulas de projeto publicadas (SMITH, 2007; ZÖLZER, 2011). A saída passa ainda por limitador para contenção de picos residuais, com ataque rápido e relaxação lenta, e um corte final no teto, porque o ganho sozinho deixa o sinal passar acima do limite na amostra em que o ataque ainda não convergiu.

Cascone fundamentou a estética da falha como material musical legítimo: erros, aliasing e artefatos digitais deixam de ser defeitos e passam a vocabulário composicional (CASCONE, 2000). O Opcoda se inscreve nessa linhagem, com a diferença de que domestica o acidente por engenharia: o glitch permanece como fonte timbrística, mas sob controle paramétrico e dentro de garantias de tempo real.

2.4 Engenharia de software em tempo real rígido

Formatos profissionais de plugin impõem contrato de tempo real rígido ao callback de processamento. O período por bloco é da ordem de milissegundos e o atraso é inadmissível, porque um estouro no callback é percebido como falha de áudio e, em muitos casos, derruba a sessão. Três proibições decorrem desse contrato, e as três são observáveis.

A primeira é nenhuma alocação dinâmica de memória durante o processamento. O alocador tem comportamento não determinístico, pode invocar o sistema operacional e pode bloquear. A segunda é nenhuma operação de entrada e saída de arquivos ou chamada de sistema bloqueante no laço prioritário. A terceira é nenhuma primitiva de sincronização com espera, porque a trava expõe o caminho de áudio à inversão de prioridade (STEINBERG, 2024; CLAP, 2024; PIRKLE, 2019).

A solução canônica separa dois domínios temporais. A interface gráfica, que roda a cerca de 60 quadros por segundo, admite alocação, entrada e saída e parsing. O motor de processamento de sinais, que opera de 44,1 kHz a 96 kHz, trabalha exclusivamente sobre memória pré-alocada. A comunicação entre os domínios utiliza fila circular sem trava de produtor único e consumidor único, baseada em operações atômicas de hardware com ordenamento acquire-release, além de parâmetros atômicos com suavização dentro do motor (PIRKLE, 2019).

Há uma consequência dessa separação que raramente é dita com todas as letras: ela não basta se a fronteira não for imposta por construção. Dizer que o motor não acessa o estado da interface é uma intenção; o que a torna verdade é o fato de não existir caminho de código que o permita, e de existir um teste que falha se alguém criar esse caminho. É essa a razão de o projeto invocar a biblioteca de framework de áudio apenas no adaptador e manter o núcleo livre dela.

3 METODOLOGIA

3.1 Abordagem e natureza da pesquisa

A pesquisa é aplicada e de desenvolvimento tecnológico, com caráter exploratório quanto às decisões de arquitetura. O objeto de estudo é um instrumento de software em construção, e o método é a implementação medida: cada decisão de arquitetura que afeta o comportamento do sistema é acompanhada por um ensaio com critério de aceite escrito antes.

Essa escolha metodológica tem uma consequência que convém explicitar. Como não há

dados de produção com usuários-alvo em escala, e como a validação perceptual exigiria estudos com ouvintes que a rotina do curso não comporta, a avaliação deste trabalho é de engenharia, e não de recepção estética. O que se mede é se o sistema cumpre contratos verificáveis: atenuação espectral, robustez diante de entrada malformada, ausência de alocação no caminho de áudio, comportamento previsível do agendador. A qualidade musical é discutida de forma breve, mas não é objeto de medição, e essa distinção aparece novamente na discussão de limitações.

3.2 Etapas de desenvolvimento

O desenvolvimento seguiu um processo de especificação antes de implementação, com constituição de projeto que define seis portões de qualidade: build, testes, tempo real, robustez, interface e documentação. Nenhuma tarefa foi encerrada sem que seu portão passasse, e a rastreabilidade entre objetivo específico, portão e ensaio é explícita em ambos os sentidos.

As etapas realizadas foram: esqueleto do projeto e parser do formato PE; motor granular, arrasto de arquivo e filtro de bloqueio de corrente contínua; notas MIDI, controle de estado e guardas; interface de três faixas, grelha de bytes e telemetria; e, por fim, congelamento e auditoria de documentação. As etapas de validação em estação de trabalho de áudio estão bloqueadas e a razão é registrada adiante.

3.3 Instrumentos de verificação

A verificação do protótipo se apoia em quatro instrumentos.

O ensaio espectral mede a atenuação da componente de corrente contínua e a energia de baixa frequência da saída do motor. Usa três binários de referência, incluindo um executável do sistema, um binário comprimido com empacotador e um arquivo sintético zerado. A captura tem dez segundos com grão de 40 ms e densidade média, e a análise usa transformada rápida de Fourier com 65536 pontos e janela Blackman-Harris. O critério é atenuação de ao menos 40 dB no bin de corrente contínua em relação à leitura bruta, e energia abaixo de 20 Hz menor que -60 dBFS. O protocolo inclui um controle negativo que renderiza a mesma conversão de byte para amostra sem motor granular e sem condicionamento, e é esse controle que dá sentido à comparação.

O ensaio de latência e carga sob polifonia mede tempo médio e percentil 99 do proces-

samento por bloco, em sessão de estação de trabalho de áudio com buffers de 128, 256 e 512 amostras a 44,1 e 48 kHz, com casos de uma, quatro e oito vozes e grãos de 10 e 100 ms. O critério é percentil 99 abaixo de 50% do orçamento, o que equivale a 2,9 ms para 128 amostras a 44,1 kHz, e zero falhas em cinco minutos contínuos.

O ensaio de estresse com binários corrompidos executa o parser em laço sobre um corpus de mutações por inversão de bit, truncamento e cabeçalhos forjados, mais dez casos de borda como arquivo vazio, arquivo de um byte e deslocamento apontando fora do arquivo. O critério é zero falha de segmentação, zero vazamento, rejeição de cem por cento das entradas inválidas com erro tipado e cobertura de ramo do parser acima de 70%.

O ensaio de aceite de uso arrasta um executável para o aplicativo autônomo e para o plugin na estação de trabalho, move os seis parâmetros principais, mapeia dois controladores de mudança de controle e toca dois minutos em cada formato. O critério é som em menos de dois segundos após o arrasto, zero falhas de tempo real e todo controlador mapeado movendo o parâmetro correspondente.

3.4 Limitações da plataforma de validação

A plataforma de desenvolvimento impõe duas limitações que precisam ser ditas antes dos resultados, e não depois.

O Windows não distribui tempo de execução de ThreadSanitizer, nem no compilador da Microsoft nem no da LLVM, e o AddressSanitizer nativo do compilador da Microsoft não cobre comportamento indefinido. O projeto ficou com um compilador só e o que ele entrega nativamente: ASan, avisos tratados como erro e testes determinísticos.

O MemorySanitizer também não está disponível, e a ausência tem peso específico. É a ferramenta que detecta leitura de memória não inicializada, e ela é exatamente a hipótese que resta para explicar um defeito descrito no capítulo de desenvolvimento: os parâmetros do plugin arrancam com valores que não são os padrões declarados. O ASan foi configurado e executa sem acusar nada, o que é coerente com essa hipótese, mas não é prova. Fechar essa lacuna exige um depurador com pontos de observação no arranque do aplicativo autônomo, e não uma correção por leitura de código.

A consequência prática sobre os ensaios é a seguinte, e é a diferença entre pendente e bloqueado que o trabalho precisa sustentar.

Ensaio	**Situação**	**Como é verificado**
Espectral	executado	Transformada rápida de Fourier com 65536 pontos e janela Blackman-Harris em teste automatizado
Latência	bloqueado	Percentil 99 medido em teste de núcleo; o ensaio com oito vozes reais exige host licenciado
Robustez	executado	Cinquenta mutações de bit e dez casos de borda, todos automatizados e com erro tipado
Aceite	bloqueado	exige instalar o bundle num host licenciado e repetir arrasto, parâmetros e controladores
ThreadSanitizer	indisponível	substituído pelo guard de alocação, que falha se o callback alocar
MemorySanitizer	indisponível	não existe para o compilador da Microsoft no Windows; o substituto é instrumentar o arranque em três pontos
Comportamento indefinido	indisponível	substituído por avisos tratados como erro e pelos testes de borda
Conformidade do bundle	indisponível	o validador trava nesta máquina; a conformidade é conferida no manifesto gerado pelo framework

Tabela 1: Situação de cada ensaio e instrumento que o substitui na ausência da ferramenta original

Elaborado pelos autores (2026).

Pendente é trabalho que está agendado. Bloqueado é trabalho que não pode ser feito com o que existe nesta máquina. As duas etapas de validação em estação de trabalho de áudio estão bloqueadas porque a única cópia do Ableton presente não é uma instalação licenciada: o diretório de programa traz um gerador de chave e um patch, e o executável arranca mas nunca abre janela utilizável, porque a ativação não está feita. Sem chave legítima não há ensaio, e ativar com o material que acompanha o pacote não é uma opção.

A troca do host de validação não afrouxou critério. O plano original nomeava dois hosts comerciais e um validador de conformidade de plugins; nenhum dos três está disponível, e o

validador trava mesmo com os argumentos de tolerância máxima. O Ableton Live 12.3.1 é o host que o usuário de fato vai usar, e fornece telemetria de latência suficiente para o critério. Trocar o host é mudança de método, não de critério: o número continua sendo percentil 99 abaixo de 50% do orçamento com zero falhas.

4 LEVANTAMENTO DE REQUISITOS

4.1 Requisitos funcionais

Doze requisitos funcionais principais foram derivados do problema de pesquisa e das Restrições de escopo. Cada um está ligado a um objetivo específico e a um ensaio do plano de validação, o que torna a rastreabilidade explícita em vez de declarada.

ID	Requisito	Objetivo	Ensaio
RF01	Aceitar arquivo por diálogo de abertura ou por arrasto do sistema operacional	OE2	T4
RF02	Validar assinatura MZ, assinatura PE, número de seções e limites por seção	OE1	T3
RF03	Converter bytes em ponto flutuante normalizado no intervalo de -1 a 1	OE1	T1
RF04	Medir entropia de Shannon por janela de 2048 bytes e por seção	OE5	T1
RF05	Sintetizar até oito vozes granulares com tamanho entre 1 e 100 ms	OE4	T2
RF06	Expor seis parâmetros automatizáveis na estação de trabalho	OE4, OE7	T4
RF07	condicionar o sinal com bloqueio de corrente contínua e limitador	OE6	T1
RF08	Controlar o motor por notas, pedal de sustentação e dois controladores de mudança de controle	OE9	T4
RF09	Mostrar a estrutura do arquivo em grelha de bytes navegável	OE7	T4
RF10	Devolver erro tipado para arquivo inválido sem interromper o material em reprodução	OE2	T3
RF11	Preservar o estado do host, incluindo parâmetros e arquivo carregado	OE7	T4
RF12	Publicar telemetria de pico, vozes ativas e entropia da janela de leitura	OE7	T2

Tabela 2: Requisitos funcionais do Opcoda e sua ligação com os objetivos específicos e os ensaios

Elaborado pelos autores (2026).

4.2 Requisitos não funcionais

Os requisitos não funcionais são o núcleo do trabalho, porque é deles que decorre a arquitetura.

O primeiro é o tempo real rígido: o callback de áudio não aloca memória, não adquire trava, não faz entrada e saída, não lança exceção e não chama o sistema operacional. Tudo o que o motor precisa está pré-alocado antes do primeiro bloco. O critério é verificável por build, e o verificador é o guard de alocação.

O segundo é a leitura defensiva: toda leitura indexada por campo de cabeçalho passa por verificação de limites, com a soma de deslocamento e tamanho conferida em aritmética que não transborda. O número de seções é limitado e o tamanho declarado de dados brutos é recortado para o que existe. O critério é zero falha de segmentação e cem por cento das entradas inválidas rejeitadas com erro tipado.

O terceiro é a não execução: o binário nunca é executado, nunca é mapeado com permissão de execução e nunca é desmontado. Nenhum ponteiro do arquivo mapeado sobrevive ao fechamento, e o motor usa apenas a cópia defensiva em memória própria.

O quarto é a preservação do núcleo: o diretório do núcleo não inclui cabeçalho de framework de áudio e o build falha se essa regra for violada. A justificativa é prática, e não estética: sanitizadores, ferramenta de fuzzing e medição de cobertura rodam diretamente sobre o núcleo, sem o framework no caminho.

O quinto é a acessibilidade da interface: operação completa por teclado, foco visível, nome acessível e valor em cada controle, contraste mínimo de 4,5 para 1, erro do parser sempre em texto com código e nunca apenas por cor.

O sexto é a consistência da documentação: cada alteração de comportamento altera a documentação no mesmo commit, e documentação desatualizada é tratada como defeito.

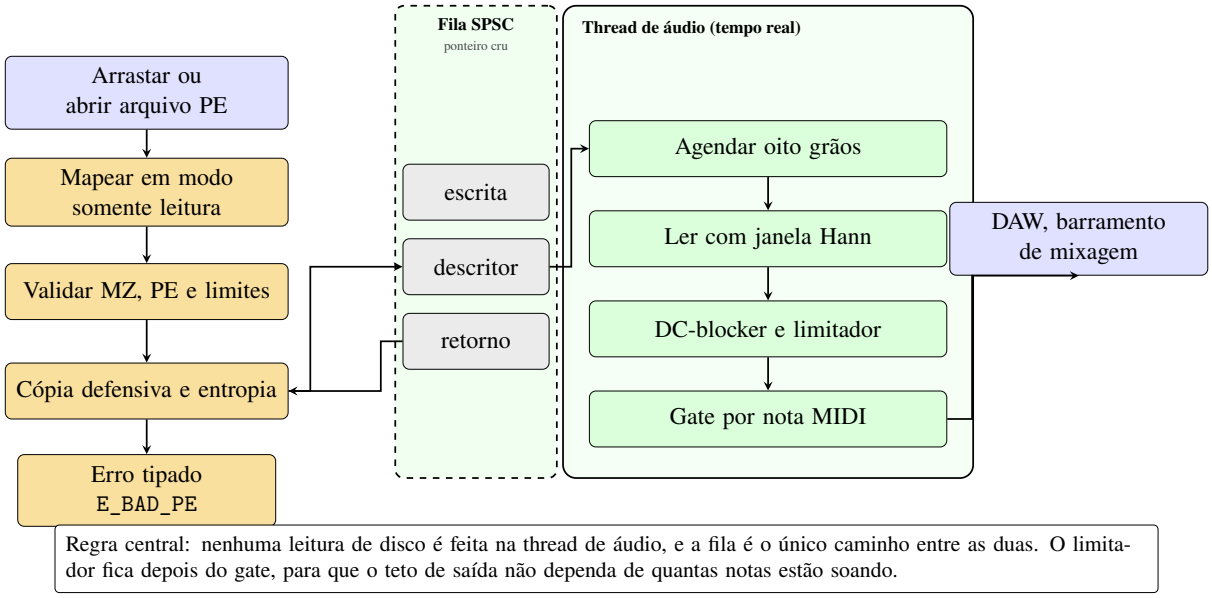


Figura 1: Pipeline de dados do Opcoda, da ingestão verificada à entrega no host
Elaborado pelos autores (2026).

5 ARQUITETURA DO SISTEMA

5.1 Visão geral e pipeline de dados

A arquitetura do Opcoda resolve um problema que tem duas naturezas distintas ao mesmo tempo: a ingestão de dados arbitrários, que pode demorar e pode falhar, e a geração de áudio, que tem prazo de milissegundos e não pode falhar. A solução é separar os dois domínios por uma fila, e tornar essa separação o único caminho possível entre eles.

O pipeline tem quatro blocos. A ingestão e o parser rodam na thread de interface: abrem o arquivo em modo somente leitura, validam assinatura e limites, copiam os dados para memória própria e calculam a entropia por janela e por seção. O bloco de troca, na fronteira de tempo real, é uma fila circular de produtor único e consumidor único, mais parâmetros atômicos. O motor granular roda na thread de áudio: agenda grãos, lê a fonte com janela, mistura as vozes e aplica o condicionamento. A entrega escreve na saída do host.

A regra que sustenta o desenho é que nada passa do parser para o motor sem cruzar a fila, e o motor nunca chama o sistema operacional. Essa regra não é uma convenção: ela é imposta pelo fato de o núcleo não conhecer a interface, e verificada por teste.

5.2 O núcleo independente de framework

A divisão do código em três partes é uma decisão de arquitetura com efeito direto sobre o que é possível verificar.

O núcleo, com 2.382 linhas de código C++20 e nenhuma dependência da biblioteca de interface, contém o parser do formato PE, a conversão de byte para amostra, a entropia de Shannon, o motor granular, as janelas, o filtro de bloqueio de corrente contínua, o limitador, a fila sem trava, o rastreador de notas e o guard de alocação.

O adaptador, com 4.965 linhas, contém o processador de áudio, os formatos e a interface. É a única parte que conhece a biblioteca de interface.

Os testes, com 3.958 linhas, exercitam o núcleo sem o framework no caminho, e é por isso que o AddressSanitizer alcança a cadeia completa, do disco até a saída de áudio, com um executável real do sistema.

Essa separação é o que permite que o ensaio de ponta a ponta exista. Ele percorre leitura de disco, parser, conversão, motor granular, filtro e limitador, sem a biblioteca de interface e sem host, e verifica que um executável real produz sinal. O ensaio exige um artefato de verdade, e não um arquivo sintético, porque é sobre material real que a divergência entre a conversão usada pelo motor e a duplicada no teste se manifesta como leitura além do fim do buffer.

5.3 O parser e a verificação de limites

O parser percorre o arquivo em quatro validações, e cada uma tem código de erro próprio. A assinatura MZ é conferida antes de qualquer campo ser lido, porque todo o resto do formato assume essa estrutura. A assinatura PE é conferida no deslocamento declarado pelo cabeçalho DOS, e esse deslocamento é validado contra o tamanho do arquivo antes de ser seguido. O número de seções é lido e limitado, e um valor acima do limite é rejeitado em vez de alocar uma tabela do tamanho que o cabeçalho pedir. Cada seção tem seu deslocamento e seu tamanho conferidos contra o tamanho real do arquivo.

Toda leitura passa por uma função única de verificação de limites, que confere a soma em aritmética de 64 bits para que a soma de deslocamento e tamanho nunca transborde antes da comparação. O tamanho declarado dos dados brutos não é confiável e é recortado para o que existe; um valor declarado maior que o disponível retorna erro de limite.

A distinção entre deslocamentos merece registro técnico, porque um erro de dois bytes

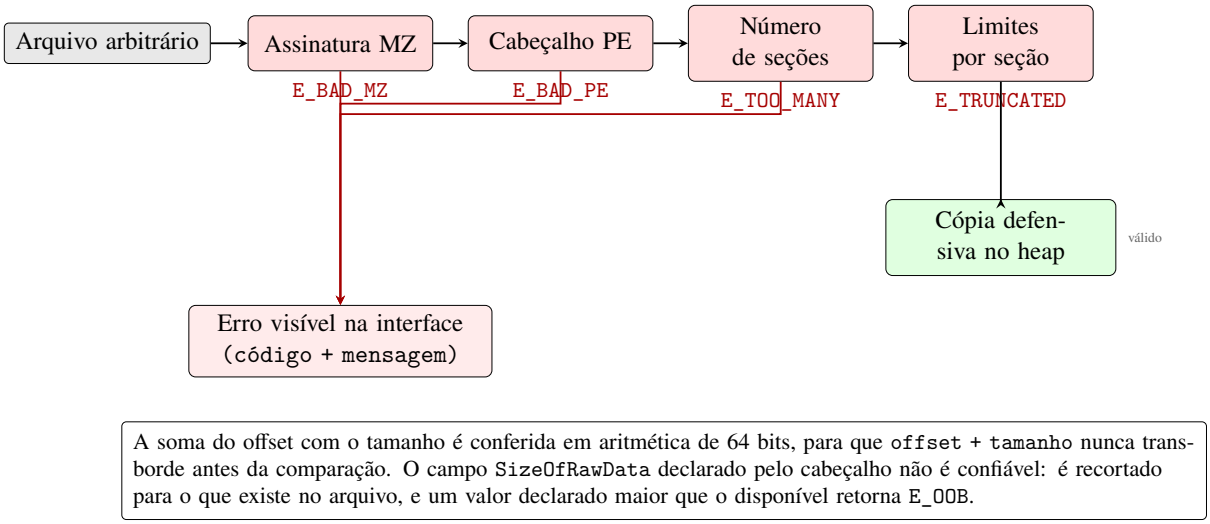


Figura 2: Cadeia de validação do parser e erros tipados correspondentes
Elaborado pelos autores (2026).

invalida todo arquivo válido sem produzir sintaxe incomumível. O cabeçalho COFF começa no deslocamento mais quatro a partir do cabeçalho PE, e não no deslocamento mais dois, que ainda pertence à assinatura de quatro bytes. O parser faz a leitura nos deslocamentos da especificação, e a suíte constrói um executável mínimo válido em memória para verificar a conformidade em vez de depender de arquivos de terceiros.

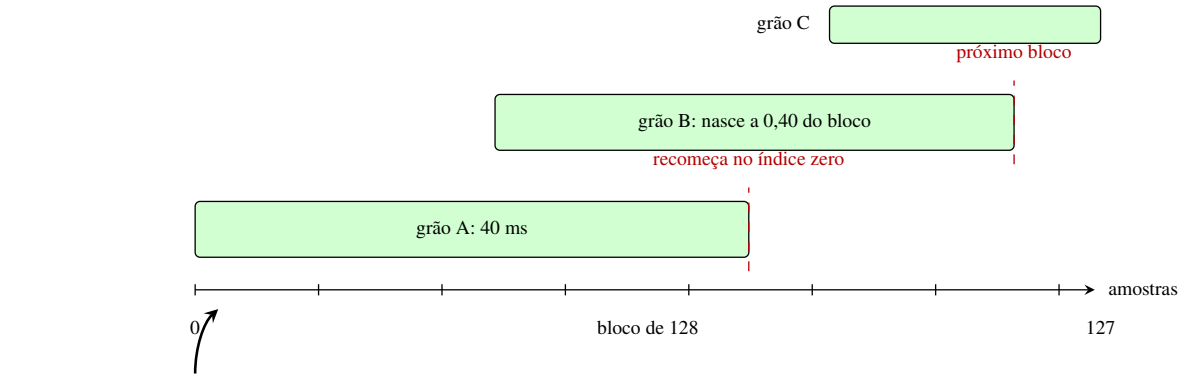
5.4 O motor granular e o agendamento

O motor mantém um conjunto fixo de oito vozes, alocado na preparação. Cada voz tem posição de leitura, taxa de avanço, índice de escrita, contador de amostra e fase de dispersão. Nenhum contêiner cresce durante o processamento.

O agendamento é onde a previsibilidade é conquistada, e vale explicar as três decisões que a tornam possível.

A primeira é a densidade como spawn por amostra, derivada dos grãos por segundo, com fração de fase preservada entre blocos. Um motor que dispara todos os grãos no início do bloco produz densidade diferente em blocos de 128 e de 512 amostras; ao manter uma fração de fase e espalhar os grãos ao longo do bloco, a densidade nominal é a mesma nos dois casos.

A segunda é o índice de escrita como posição dentro do bloco corrente, e não como contador global de amostras. Um grão que atravessa o bloco recomeça em zero no índice de escrita; sem esse reset, a voz entra no bloco seguinte com índice maior que o tamanho do bloco e nunca mais escreve, o que produz silêncio sem erro.



fração de fase preservada entre blocos de 128 e 512

Cada grão guarda o índice de escrita onde nasceu, porque a sobreposição precisa começar na amostra certa. O índice é a posição dentro do bloco corrente, não um contador global: sem esse reset a voz trava e nunca mais escreve. Todo grão começa com a janela no índice zero, o que zera a amplitude nas bordas e evita o clique na sobreposição.

Figura 3: Agendamento dos grãos dentro de um bloco de 128 amostras
Elaborado pelos autores (2026).

A terceira é o avanço por amostra como a taxa de reprodução dividida pelo comprimento da fonte, porque a posição é normalizada no intervalo de zero a um. A taxa bruta não pode ser somada à posição normalizada, porque o grão percorreria a fonte inteira em duas amostras. A asserção de pico maior que zero não detecta esse erro, o que motivou teste próprio de agendamento com verificação de nível absoluto, e não apenas de razão entre condições de referência.

O interpolador é linear, com passo limitado para impedir aliasing audível na afinação máxima de mais vinte e quatro semitons. Todo grão começa com a janela no índice zero, o que zera a amplitude nas bordas e evita o clique na sobreposição. A entropia local modula a dispersão: região de alta entropia espalha mais, região repetitiva fica mais ancorada.

5.5 A fronteira de tempo real e a troca de material

A comunicação entre interface e motor usa duas filas em sentidos opostos, e não uma. A interface publica o ponteiro do novo material e guarda o buffer em um contêiner próprio; a thread de áudio troca o ponteiro ativo e devolve o antigo pela fila de retorno. Só a interface remove o buffer do contêiner, e só depois que a thread de áudio confirmou que terminou de

usá-lo.

Uma fila só seria mais curta e incorreta: sem a volta, a interface não sabe quando é seguro liberar, e liberar cedo é uso após liberação. O teste que cobre essa invariante verifica que o buffer antigo só é liberado depois que o motor passou adiante.

A fila carrega ponteiro cru, nunca ponteiro compartilhado, por duas razões que apontam para a mesma escolha. A fila exige tipo trivial para copiar o slot sem contagem de referência, e decrementar um contador na thread de áudio pode executar o operador de liberação ali dentro, o que é violação de tempo real.

Um duplo tampão com troca atômica existiu no projeto e foi removido. A dupla fila resolve o mesmo problema com menos peças, reaproveitando a fila circular que já estava testada, e um teste com duas threads de verdade cobre a invariante.

5.6 A interface e a grelha de bytes

A interface tem três faixas, na convenção do Ableton: cabeçalho claro, display escuro e painel de parâmetros. A escolha tem motivo funcional: halo de brilho sobre superfície clara desaparece, com contraste de 2,4 para 1, de modo que toda a energia de brilho fica na região escura.

Os controles giratórios são componentes reais de deslize do framework, e não desenho customizado. A decisão é deliberada: o componente entrega foco por teclado, ajuste com setas e manipulador de acessibilidade de graça, que são exatamente os critérios de acessibilidade adotados. Desenhar o controle à mão custaria tudo isso. O nome acessível de cada controle é a descrição completa em português, e não o rótulo curto do protótipo, que seria inútil para quem navega por leitor de tela.

O display é uma grelha de bytes no formato de um despejo hexadecimal: endereço à esquerda, dezesseis bytes por linha e coluna de caracteres à direita. A faixa anterior era um mapa do arquivo inteiro com cursor, e um arquivo de doze megabytes não cabe em grelha; o que a grelha dá em troca é o byte.

O clique escreve o parâmetro de posição, porque escolher um byte é escolher de onde se lê. Se o byte estiver fora da região em reprodução, o editor puxa a região para lá antes de escrever o parâmetro, e a ordem não é arbitrária: a posição é uma fração da região, e escrevê-la antes de a região mudar apontaria para o sítio errado. O comprimento da região preserva-se.

O mapeamento entre endereço e fração vive no núcleo, e não no componente da grelha,

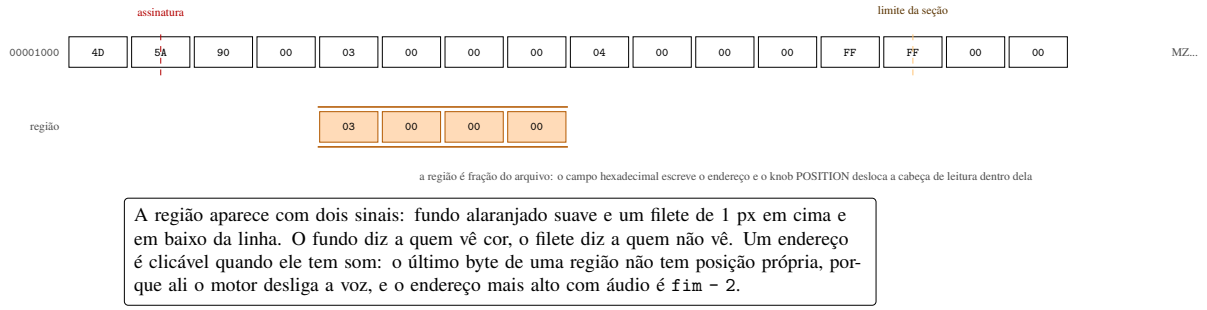


Figura 4: Grelha de bytes com a região de leitura em reprodução
Elaborado pelos autores (2026).

pelo mesmo motivo da região: o núcleo é o que os sanitizadores alcançam. Ele tem duas armadilhas que estão em teste. O último byte não tem posição própria, porque ali o motor desliga a voz, e o endereço mais alto com som é o fim menos dois. E o caminho inverso arredonda, em vez de truncar: truncar punha o cursor sistematicamente um byte à esquerda do que foi clicado, em sessenta e três endereços de mil em um teste, enquanto arredondar ao mais próximo centra o erro em zero.

5.7 Notas, controladores e estado do host

O rastreador de notas está no núcleo, sem framework, pelo mesmo motivo da conversão de byte para amostra: é a regra que tem testes. O modelo separa teclas premidas de notas presas pelo pedal de sustentação, porque com o pedal premido largar o teclado não desliga o som, e soltar o pedal não abafa as teclas que continuam premidas. Um contador único não consegue expressar as duas condições.

Há um piso em zero na contagem. Uma nota desligada sem nota ligada correspondente vem de hosts que reenviam o estado inicial, e sem esse piso o estado de som ficaria falso para sempre: o instrumento carregava, o controle girava, e nunca mais saía som sem reiniciar o plugin.

Os dois controladores mapeados são o de brilho, mapped para densidade, e o de ressonância, mapped para posição. São os dois com nome correspondente em qualquer teclado de palco, então não precisam de tabela para serem descobertos. O valor lido no callback vai para o rastreador e é aplicado ao parâmetro na thread de interface, a partir do temporizador do editor. O caminho alternativo, escrever direto no parâmetro da thread de áudio, foi descartado de propósito: notificar o host dispara a cadeia de ouvintes, e o vínculo do controle com o parâmetro

é um deles, o que seria acesso cruzado à interface a partir do callback, exatamente a classe de defeito que o guard de alocação não apanha. O custo é a latência do temporizador, cerca de cinquenta milissegundos. Em troca, o parâmetro continua sendo a fonte única da verdade.

O gate de notas entra antes do limitador, para que o comportamento do limitador não dependa de quantas notas estão soando. A rampa é linear e dura cinco milissegundos, e isso não é detalhe estético: abrir a saída de uma vez produz um degrau no sinal, e um degrau é um transiente largo em frequência, audível mesmo com libertação curta.

O estado do host é preservado com o caminho que o próprio host usa para empurrar parâmetros de automação de volta para dentro, e não com uma via paralela. O arquivo carregado entra no estado como propriedade da mesma árvore, pelo mesmo caminho do arrasto.

6 IMPLEMENTAÇÃO

6.1 Ferramentas e tecnologias

A implementação usa C++20 com a biblioteca JUCE 8.0.14, que fornece a abstração de processador de áudio, os formatos de plugin e a interface de usuário. O sistema de build é CMake com gerador NMake, e o compilador é o Visual Studio Build Tools 2026, toolset v145. O framework de testes é o GoogleTest, obtido por download automático na configuração. O SDK do VST3 vem embutido na própria biblioteca, e não há dependência externa versionada no repositório.

A biblioteca JUCE é restrita ao adaptador. O núcleo compila como biblioteca C++20 pura, e a regra é verificada por build: se alguém incluir um cabeçalho do framework no núcleo, a compilação falha.

6.2 Cadeia de sinal e conversões

A cadeia de sinal tem quatro etapas: bytes viram ponto flutuante normalizado, o motor granular sobrepõe as vozes, o filtro de bloqueio de corrente contínua remove o deslocamento, e o limitador contém os picos.

A conversão de byte para amostra é a fronteira entre o binário e o motor, e ela vive no núcleo, e não na interface nem no teste, para que o plugin e o ensaio espectral chamem a mesma função. A normalização coloca o valor 127,5 no zero e leva os extremos a menos um e mais um.

Esse deslocamento de meio nível é justamente o que produz o problema que o filtro remove, e é por isso que o ensaio testa o filtro com sinal constante.

A conversão tem um único endereço no código, e o ensaio espectral e o plugin chamam a mesma função. A duplicação anterior da conversão dentro do teste é a origem de uma leitura além do fim do buffer detectada pelo AddressSanitizer, o que fixa a regra de que a fronteira entre forma de onda e domínio de teste tem implementação única.

O filtro de bloqueio de corrente contínua usa a recorrência descrita na fundamentação, com coeficiente de 0,9983, que corta em cerca de 12 Hz a 44,1 kHz.

A divergência corrigida durante o projeto merece registro, porque é o tipo de erro que sobrevive à revisão. A fundamentação, o artigo e os documentos de objetivos e arquitetura indicavam coeficiente 0,995 com corte entre 10 e 15 Hz. Os dois números não são compatíveis: 0,995 corta em 35,2 Hz a 44,1 kHz. O código usa o valor que produz o corte citado, porque o ensaio espectral mede a faixa de frequência e não o coeficiente, e o texto foi corrigido nos quatro lugares com a relação explícita.

O limitador suaviza o ganho de forma assimétrica, com ataque rápido e relaxação lenta. O ganho sozinho deixa o sinal passar acima do teto na amostra em que o ataque ainda não convergiu, e por isso há um corte final. É a troca clássica de limitador sem antecipação: garante o teto ao custo de alguns decibéis de distorção no ataque. Sem o corte, o limitador não limita, e o teto é exatamente o que o ensaio espectral mede.

6.3 Entropia por seção

A medição de entropia por seção exige comprimento explícito. A sobrecarga que recebe dados, tamanho e deslocamento mede do deslocamento até o fim do arquivo, o que é incorreto para uma seção: o fim da seção é o deslocamento mais o tamanho, e esse tamanho é o mínimo entre o declarado e o disponível, de modo que costuma ser menor que o resto do arquivo. A implementação mantém as duas formas, e a de quatro argumentos, com comprimento explícito, é a chamada usada pela ingestão.

Seção sem dados brutos, que é o caso da seção de dados não inicializados, tem tamanho bruto zero e fica com entropia zero. Medir a partir do deslocamento declarado devolveria a entropia do material seguinte, que não responde à pergunta feita sobre a seção.

6.4 Guard de alocação

A restrição de tempo real é transformada em código por substituição do operador global de alocação por uma versão que conta alocações dentro da janela de áudio. Um teste marca a janela e falha se o processamento de bloco alocar, inclusive em cinco minutos de reprodução sintética.

Isso é o critério de tempo real virando código, e não revisão manual. Oito casos de teste cobrem o guard.

Há uma lacuna que o guard não cobre e que precisa ser dita: ele roda sobre o núcleo e não vê o processamento de bloco do plugin, onde as mensagens MIDI são lidas. A leitura de notas é uma passagem por um buffer de mensagens já contadas, sem alocação previsível, mas isso é raciocínio e não medição. O que fecha essa lacuna é o ensaio de latência com tráfego MIDI real, que depende de host licenciado.

6.5 A interface e os defeitos de layout

Cinco arquivos, cada um com uma responsabilidade só, implementam a interface: a paleta com os tokens visuais extraídos do protótipo, o desenho do controle e do botão, o indicador luminoso, o rótulo com fundo, o painel de display e a grelha de bytes. Nenhum deles contém lógica de áudio.

Quatro defeitos que só a captura de tela apanhou merecem registro, porque nenhum aparece em teste e nenhum é de lógica: são medidas.

O primeiro foi a cor de texto do rodapé, herdada da superfície clara e ilegível sobre o display escuro, o que reprova o critério de contraste. O segundo foi o empilhamento do rodapé, em que todas as caixas ocupavam o mesmo intervalo porque a posição de cada uma era medida a partir da margem esquerda a cada passo, e só a última pintada aparecia. O terceiro foi a moldura das abas e das leituras vazias desenhando um retângulo sem nada dentro quando não há arquivo. O quarto foi a área da grelha invadindo o rodapé, por receber a coordenada absoluta do rodapé no lugar de uma altura, o que com a janela baixa produzia texto ilegível empilhado.

Dois deles têm lição de processo. Uma caixa vazia que parece defeito deve se esconder, e um alvo que salta ao carregar um binário é pior do que um espaço constante. E uma coordenada não é um tamanho: o nome do parâmetro é o que impede o erro.

Houve ainda três decisões de interface que deram errado na primeira tentativa e estão registradas: o método que deveria consumir o retângulo não o consome, e o layout passou a

usar uma coordenada explícita; não existe modo de mistura aditivo na biblioteca, e o brilho do indicador passou a ser um gradiente radial com queda de transparência; e quebrar os controles em duas linhas não funciona, porque a altura disponível não comporta dois controles com rótulo e valor.

6.6 Correções de documentação industrial

O projeto adota a regra de que documentação divergente do código é defeito, e a auditoria encontrou divergências reais. A mais visível foi o coeficiente do filtro, já descrita. Otras foram o biquad de coloração declarado no artigo sem parâmetro correspondente, a transformada rápida de Fourier listada como dependência do plugin quando existe apenas na ferramenta de medição, e a contingência de troca de biblioteca de interface que deixou de ser necessária.

Essas correções são registradas em tabela de decisões no apêndice, e a regra que as produz é a mesma: cada alteração de comportamento altera a documentação no mesmo commit.

7 VERIFICAÇÃO E RESULTADOS

7.1 Suíte de testes automatizados

A suíte de verificação reúne 196 casos em 24 conjuntos, mais 8 casos do guard de alocação. Todos os casos passam na configuração de release. A distribuição por área, conforme a execução real da suíte, é a seguinte.

Área funcional	**Casos**	**O que cobre**
Parser do formato PE	21	assinatura MZ e PE, limites, truncamento, número de seções, mutações por inversão de bit
Entropia de Shannon	11	símbolo repetido, janela com deslocamento, comprimento explícito por seção
Motor granular e janelas	20	agendamento, dispersão, gate por nota, interpolação, janelas Hann, Hamming e Blackman
Filtro de corrente contínua e limitador	14	corte na faixa prometida, sinal constante, rampa, teto, reinicialização
Fila e troca de material	15	produção e consumo, invariante de liberação, parâmetros atômicos, retrocesso
Posição e região	57	mapeamento de endereço para fração e o inverso, ajuste à seção, redução a colunas, limites de faixa
Transporte e visualização	25	reprodução, seleção de seção, estado de linha e de transporte
Rastreio de notas	10	sustentação, acordes, nota órfã, controladores, reinicialização
Cadeia completa	9	disco, parser, conversão, motor, filtro e limitador
Ensaio espectral e conversão	14	atenuação de corrente contínua, banda abaixo de 20 Hz, faixa de byte normalizada
Guard de alocação	8	janela de áudio, liberação, alocação dentro e fora da janela

Tabela 3: Distribuição dos casos de teste por área funcional, conforme execução da suíte em configuração de release

Elaborado pelos autores (2026), a partir da execução da suíte de testes em configuração de release.

O ensaio end-to-end percorre a cadeia completa com um executável real do sistema, sem a biblioteca de interface e sem host. O executável de teste da máquina tem duzentos quilobytes, sete seções e byte médio de 96 na seção de código, o que corresponde a um deslocamento de menos 0,245 em amostra normalizada: material que o filtro de bloqueio de corrente contínua

tem o que remover e que a leitura sintética não reproduzia.

7.2 Resultado do ensaio espectral

O ensaio de atenuação é executado e cumpre o critério. O primeiro caso compara o bin de corrente contínua da saída tratada com o mesmo bin da leitura bruta, em um executável sintético cujo conteúdo tem média muito distante de zero. A asserção exige atenuação de ao menos 40 dB, e o teste passa. O segundo caso exige que o pico abaixo de 20 Hz fique abaixo de menos 60 dBFS, e o teste passa.

O terceiro caso isola a cadeia: um sinal puramente constante é submetido ao filtro, e a energia quadrática média da saída, medida depois de um período de acomodação, fica abaixo do limiar. A medição é feita depois da acomodação porque a transitória de partida decai com a potência do coeficiente, e o regime permanente é o que interessa.

O quarto a sétimo casos verificam a conversão de byte para amostra: o mapeamento do intervalo de bytes para o intervalo de menos um a mais um, o respeito ao deslocamento, a recusa de entrada degenerada e a energia mínima nas bordas da janela.

O critério de atenuação é uma asserção de limite inferior, e um sinal cujo pico seja da ordem de dez elevado a menos sete a satisfaz sem que exista som audível. O ensaio mede a razão entre duas condições de referência e não o nível absoluto da saída, o que o torna insensível à perda total de sinal; a correção foi tornar o agendamento observável por testes próprios com verificação de nível absoluto, e não reformular o critério de razão.

7.3 Resultado do ensaio de robustez

Os dez casos de borda do plano de validação estão automatizados e passam. Cobrem arquivo vazio, arquivo de um byte, deslocamento apontando fora do arquivo, número de seções acima do limite, tamanho declarado de dados brutos maior que o disponível, seções com deslocamento fora do arquivo, assinatura MZ válida com assinatura PE ausente, tamanho de cabeçalho opcional inconsistente, entropia sobre seção sem dados e truncamento no meio de uma seção.

Todos produzem erro tipado, sem exceção e sem falha de segmentação. O corpus de cinquenta mutações por inversão de bit previsto no plano foi escrito como casos determinísticos, e cada mutação declara o erro que se espera dela, de modo que uma mudança de comportamento

no parser que passasse a devolver outro código quebra o ensaio. A varredura é fixa e não exaustiva: cobre os campos que o leitor consulta e não todo o espaço de entradas, pelo que a rejeição de entrada malformada está demonstrada para o corpus e não para todas as sequências de bytes possíveis.

O portão de cobertura de ramo acima de 70% não tem medição automática, porque a ferramenta de cobertura de ramo não está disponível no toolchain escolhido. Os testes exercitam os caminhos de erro de forma explícita, mas o número não é medido, e o texto diz isso.

7.4 Resultado do ensaio de latência

O ensaio de latência está bloqueado e não executado. O número não existe porque exige uma estação de trabalho de áudio licenciada, e a única cópia presente na máquina de desenvolvimento não é uma instalação válida.

O que se pode afirmar sem host é que o percentil 99 do tempo de processamento é medido em teste de núcleo, e que o guard de alocação passa. Isso cobre a fração estática do contrato de tempo real, e não cobre a fração dinâmica, que é a que o ensaio com host mediria. A distinção é mantida: o que não foi medido não é afirmado.

7.5 Resultado do ensaio de aceite

O ensaio de aceite está bloqueado pelo mesmo motivo. O que já foi verificado sem estação de trabalho é que o manifesto do bundle declara as categorias de instrumento e sintetizador, e que o carregamento da biblioteca do plugin foi exercitado em host real quando o plugin foi aceite em uma sessão anterior. O que falta é a telemetria de latência com tráfego real e a confirmação de que todo controlador mapeado move o parâmetro correspondente.

7.6 Defeito conhecido e não corrigido

Há um defeito conhecido, visível no aplicativo autônomo e não corrigido, que o trabalho deve reportar com honestidade.

Os seis parâmetros arrancam com valores que não são os padrões declarados no descritor de layout. As caixas de valor mostram o que fica: 1 ms, cerca de 29 por segundo e 69 por cento,

onde os padrões são 40 ms, 20 por segundo e 50 por cento. Foi confirmado em compilação separada, logo não vem de nenhum componente de interface.

A instrumentação por escrita em arquivo de texto em três pontos estabeleceu o seguinte, em valores normalizados, que é o que o método de leitura devolve nesta biblioteca.

Ponto	Tamanho de grão	Densidade	Posição
Fim do construtor do processador	0,393939	0,095477	0,500000
Entrada da ingestão	1,000000	0,060000	0,688000

Tabela 4: Valores dos parâmetros normalizados em três pontos do arranque do aplicativo autônomo

Elaborado pelos autores (2026).

Na saída do construtor os valores estão certos, que é o que o mecanismo de parâmetro deve fazer. Já à entrada da ingestão estão errados, e a ingestão não escreve parâmetros. A escrita acontece no arranque do aplicativo autônomo, entre o construtor e a primeira ingestão, sem editor e sem temporizador, o que elimina a aplicação de mudanças de controlador, que é chamada pelo temporizador do editor.

Os outros três parâmetros só parecem certos: têm padrão zero, e memória zerada por acaso dá o mesmo resultado. Estão corrompidos como os outros.

O AddressSanitizer não acusa nada. Com o plugin compilado em configuração separada com o sanetizador de endereços ativo, a arrancar e a carregar um binário, a saída de erro fica vazia. A primeira tentativa falhou e parecia que o sanetizador não funcionava, porque o aplicativo arrancava sem janela; a causa era o tempo de execução dinâmico do sanetizador, que vive ao lado do compilador e não estava no caminho de busca do processo.

Descartados por leitura, todos protegidos: a fila tem índices limitados; o rastreador de notas verifica limites em controlador e ignora eventos fora de faixa; os parâmetros atuais usam a leitura do valor bruto, que dá o valor guardável; a leitura de notas mapeia somente dois controladores para dois índices; a publicação de telemetria escreve quatro atômicos; e o preenchimento da janela respeita o comprimento e recusa valores não positivos.

O que resta é leitura de memória não inicializada, que o sanetizador de endereços não vê e que o de memória veria. Como o de memória não existe para o compilador da Microsoft no Windows, fechar essa lacuna exige um depurador com pontos de observação no arranque, e não uma correção por leitura de código. O defeito não foi corrigido aqui porque mexer às cegas seria trocar um sintoma por outro.

8 DISCUSSÃO

8.1 O que foi confirmado

A hipótese do trabalho se confirma nos seus três componentes verificáveis, com uma ressalva importante sobre o segundo.

A atenuação da corrente contínua é confirmada por ensaio automatizado com critério de aceite: a saída tratada apresenta o bin de corrente contínua pelo menos 40 dB abaixo do mesmo bin da leitura bruta, e a energia abaixo de 20 Hz fica abaixo de menos 60 dBFS. Esse é o componente que dependia da qualidade do tratamento de sinal, e a janela com bordas nulas mais o filtro de primeira ordem bastam para cumpri-lo.

A robustez diante de entrada malformada é confirmada nos dez casos de borda automatizados, todos com erro tipado e sem falha de segmentação. A prática de leitura defensiva com verificação de limites, sem executor o binário, é o que torna isso possível.

A restrição de tempo real é confirmada no núcleo: o guard de alocação passa, inclusive em cinco minutos de reprodução sintética, e a fila sem trava com ponteiro cru cobre a fronteira entre os dois domínios. A ressalva é que o guard não cobre o processamento de bloco do plugin, e a validação dinâmica está bloqueada por ausência de host licenciado.

8.2 O que não foi confirmado

A fração dinâmica do contrato de tempo real não foi medida, e o ensaio de aceite com o usuário não foi executado. Ambos estão bloqueados, não atrasados, e a diferença é registrada em todo o texto.

O portão de cobertura de ramo acima de 70% não tem medição automática, porque a ferramenta de cobertura de ramo não está disponível no toolchain de compilador único que a plataforma impôs.

O defeito dos parâmetros que arrancam errados está aberto, com a hipótese de leitura de memória não inicializada registrada e a ferramenta que a confirmaria indisponível.

A qualidade musical do instrumento não foi objeto de medição. O trabalho avalia contratos de engenharia, e a audibilidade do material gerado é uma afirmação qualitativa que não substitui o ensaio com ouvintes.

8.3 Decisões de arquitetura e seus custos

Vale registrar as decisões que foram tomadas com o custo, porque elas definem o caráter do trabalho.

A primeira é o núcleo livre de framework. O custo é uma camada de indireção entre o núcleo e o plugin, com tradução explícita de tipos. O benefício é mensurável: sanitizadores e a ferramenta de fuzzing rodam sem o framework no caminho, e o ensaio end-to-end existe.

A segunda é o compilador único. O plano previa dois compiladores e o projeto ficou com um, o que entregou sanetizador de endereços nativo, avisos tratados como erro e testes determinísticos, e perdeu o sanetizador de threads e o de memória. A escolha é defensável para o objetivo, mas o custo de perder a detecção de leitura não inicializada foi real, e foi o que deixou o defeito dos parâmetros sem confirmação.

A terceira é o duplo tampão substituído por dupla fila. A primeira versão era mais simples de descrever, e a segunda tem menos peças e reaproveita uma estrutura já testada.

A quarta é o host de validação trocado. O plano original nomeava dois hosts comerciais e um validador de conformidade; o validador trava nesta máquina e o Ableton é o host que o usuário de fato vai usar. A troca não afrouxou critério, apenas mudou de método.

A quinta é o escopo. Oito vozes, seis parâmetros, Windows, dois formatos. Nada disso foi antecipado. Um instrumento com dezesseis vozes e modulação matricial seria mais impressionante e menos verificável, e a escolha foi pela verificabilidade.

8.4 Limitações

As limitações do trabalho se dividem em três grupos.

A primeira é da plataforma de validação: ausência de ThreadSanitizer e de MemorySanitizer no Windows com o compilador disponível, ausência de estação de trabalho de áudio licenciada e ausência de ferramenta de cobertura de ramo automática.

A segunda é do escopo: apenas oito vozes, apenas janelas Hann e gaussiana, apenas Windows x64, sem reconhecimento de assinaturas de empacotadores e sem modulação por rede.

A terceira é da natureza da avaliação: a qualidade musical não foi medida com ouvintes, e a validação de usabilidade com escala SUS está prevista no plano mas não executada.

9 CONCLUSÃO

Este trabalho entregou um instrumento que transforma arquivos executáveis em som dentro de uma estação de trabalho de áudio, e a entrega se sustenta em três afirmações verificadas.

A primeira é que a leitura estática verificada de um artefato arbitrário pode ser feita dentro de um instrumento sem expor o host a risco. O parser valida assinatura, número de seções e limites de cada seção, recorta tamanhos declarados que não existem, recusa entrada degenerada com erro tipado e nunca executa o binário. Nos dez casos de borda do ensaio de robustez, todos produziram erro tipado sem falha de segmentação.

A segunda é que o tratamento de sinal converte material inadequado em material utilizável, com número medido. O janelamento granular com bordas nulas mais o filtro de primeira ordem reduzem o bin de corrente contínua em pelo menos 40 dB em relação à leitura bruta, e levam a energia abaixo de 20 Hz abaixo de menos 60 dBFS. O ensaio inclui controle negativo, que é a mesma conversão sem motor e sem condicionamento, e é ele que dá sentido à comparação.

A terceira é que a disciplina de tempo real pode ser imposta por construção e verificada por build. A separação entre o núcleo, que não conhece a interface, e o adaptador, que conhece o framework, torna impossível o acesso do motor ao estado da interface sem criar um caminho novo. O guard de alocação falha o build se o caminho de áudio alocar, e passa em cinco minutos de reprodução sintética.

As limitações também foram verificadas e são parte do resultado. A validação dinâmica de tempo real e o ensaio de aceite com o usuário estão bloqueados por ausência de estação de trabalho de áudio licenciada, e essa lacuna está escrita em vez de omitida. A ferramenta que detectaria leitura de memória não inicializada não existe para o compilador disponível no Windows, e por isso o defeito dos parâmetros que arrancam com valores errados fica com hipótese registrada e sem confirmação. A cobertura de ramo do parser não é medida por falta de ferramenta.

O trabalho também produziu evidência sobre um ponto que costuma ser tratado como preferências de estilo. A separação entre domínio de interface e domínio de processamento, com fila sem trava e ponteiro cru na fronteira, não é uma declaração de intenção: é uma estrutura que os sanitizadores alcançam, e é o que permitiu que o ensaio de ponta a ponta existisse e que os defeitos de maior impacto fossem encontrados por ensaio e não por acaso em sessão de uso.

9.1 Trabalhos futuros

O primeiro caminho é o fechamento das etapas bloqueadas, que exige obter uma estação de trabalho de áudio licenciada e executar os ensaios de latência e de aceite com oito vozes reais e tráfego MIDI real. Sem esse passo, a fração dinâmica do contrato de tempo real permanece não medida.

O segundo é o formato CLAP, previsto na arquitetura e fora da entrega do MVP, que dá portabilidade para Linux e macOS sem recompilar o núcleo.

O terceiro é o corpus de mutações por inversão de bit, que transforma o ensaio de robustez de dez casos de borda em fuzzing de fato, com a ferramenta de fuzzing disponível em outro toolchain.

O quarto é a tinta de entropia por linha na grelha, que codifica a entropia da janela de leitura como cor de fundo da célula. É o passo natural a seguir, porque a entropia por janela já é calculada e a grelha já tem a linha, e ele fica de fora por uma razão de acessibilidade: cor de fundo que codifica um número é o primeiro sinal a falhar o critério de uso da cor, e por isso precisa de uma segunda leitura, não só de um alfa.

O quinto é fechar o defeito dos parâmetros que arrancam errados, o que exige um depurador com pontos de observação no arranque do aplicativo autônomo, e não nova leitura de código.

9.2 Considerações finais

A reflexão que o trabalho sugere é sobre o custo da disciplina. Toda a arquitetura do Opcoda é mais complicada do que o mínimo necessário para ler bytes e tocar som, e cada complicação existe porque uma regra foi escrita antes do código. A fila sem trava em vez de uma chamada direta, o núcleo separado do framework, o parser com quatro validações e erros tipados, o guard de alocação, os seis portões de qualidade: nenhum disso é necessário para produzir som, e todos são necessários para produzir som dentro do processo de outro programa, onde um erro não é um erro, é uma sessão de trabalho perdida.

O outro lado dessa disciplina é que ela cobra. O trabalho registrou correções de documentação divergente do código, entre elas um coeficiente de filtro incompatível com a frequência de corte declarada, um defeito de agendamento que o critério de aceite não detectava e um defeito de arranque que a ferramenta disponível não alcança. A lição prática é que a lista de ferramentas indisponíveis integra o método e não constitui nota de rodapé.

A recomendação que o trabalho deixa é de método, não de ferramenta: em sistemas que precisam falhar de forma previsível, o tratamento de entrada arbitrária não é um detalhe de robustez, é a parte da arquitetura que decide se o sistema pode ser usado por alguém além de quem o escreveu. No Opcoda, essa parte ficou explícita desde o primeiro objetivo específico, e é o que permitiu que o resto do sistema evoluísse sem medo.

APÊNDICES

APÊNDICE A , Códigos de erro do parser e sua origem

Código	**Condição detectada**	**Onde é verificado**
E_BAD_MZ	Assinatura MZ ausente nos dois primeiros bytes	Primeiro teste do parser
E_BAD_PE	Assinatura PE ausente no deslocamento declarado	Segundo teste do parser
E_TRUNCATED	Arquivo menor que o deslocamento do cabeçalho PE	Caso de borda
E_OOB	Deslocamento mais tamanho maior que o tamanho do arquivo	Caso de borda
E_TOO_MANY_SECTIONS	Número de seções acima do limite de 96	Caso de borda
E_SOURCE_MISSING	Arquivo gravado no estado do host não existe mais	Teste de estado

Tabela 5: Códigos de erro tipados devolvidos pela ingestão e a condição que os produz
Elaborado pelos autores (2026).

Nenhum desses códigos propaga exceção através da interface binária do plugin. A entrada inválida sempre resulta em retorno de erro, e o material que estava em reprodução antes continua tocando.

APÊNDICE B , Matriz de rastreabilidade entre objetivo, portão e ensaio

Objetivo	Portão	Ensaio	Situação
OE1, parser com verificação de limites	B, D	T1, T3	executado
OE2, ingestão somente leitura com erros tipados	D	T3	executado
OE3, fila sem trava entre interface e motor	C	T2	núcleo verificado, ensaio com host bloqueado
OE4, motor granular de oito vozes	B, C	T1, T2	núcleo verificado, ensaio com host bloqueado
OE5, entropia por janela e por seção	B	T1	executado
OE6, condicionamento com filtro e limitador	B	T1	executado
OE7, interface com grelha e parâmetros automatizáveis	E	T4	bloqueado
OE8, isolamento entre interface e motor	C	T2	guard de alocação verde, ensaio com host bloqueado
OE9, mapeamento MIDI nativo	B, E	T4	núcleo verificado, ensaio com host bloqueado

Tabela 6: Rastreabilidade entre objetivos específicos, portões de qualidade e ensaios

Elaborado pelos autores (2026).

APÊNDICE C , Divergências de documentação corrigidas

Onde	O que estava errado	Situação
Fundamentação, artigo, objetivos e arquitetura	Coeficiente do filtro declarado de forma incompatível com a frequência de corte prometida	corrigido nos quatro lugares, com a relação explícita
Artigo, seção de parâmetros	Biquad de coloração descrito sem parâmetro correspondente	pendente de decisão entre estágio fixo e sétimo parâmetro
Objetivos e arquitetura	Biquad ausente da lista de etapas	dependente da decisão acima
Documento de arquitetura	Transformada rápida de Fourier listada como dependência do plugin	removida, existe apenas na ferramenta de medição
Cronograma	Contingência de troca de biblioteca de interface	desativada, a biblioteca compila no toolset corrente
Manifesto do bundle	Categoria de instrumento incorreta	corrigida, o bundle declara instrumento e sintetizador
Processador de áudio	Barramento de entrada estéreo declarado e nunca usado	removido, instrumento sem entrada
Estrutura da imagem do PE	Campo de entropia por seção declarado e nunca preenchido	corrigido, medido na ingestão

Tabela 7: Divergências entre documentação e código, com a situação de cada uma

Elaborado pelos autores (2026).

APÊNDICE D , Estrutura do código e volume verificado

Diretório	Linhas	Responsabilidade
Núcleo, código C++20	2.382	parser, conversão, entropia, motor granular, filtro, limitador, fila, rastreador de notas
Adaptador, código com framework	4.965	processador de áudio, formatos, interface de três faixas, grelha de bytes
Testes	3.958	196 casos em 24 conjuntos, mais 8 do guard de alocação

Tabela 8: Distribuição do código verificado do projeto

Elaborado pelos autores (2026).

O volume de teste é maior que o do núcleo, e isso é deliberado: no núcleo, quase toda linha

executável decide se um arquivo arbitrário é aceito ou rejeitado, e a verificação correspondente é a parte que não se negocia.

Referências

AUDACITY TEAM. *Import raw data: the manual of the Audacity audio editor*. [S.l.], 2026. Disponível em: <https://manual.audacityteam.org/m/man/importrawdata.html>.

CASCONE, K. The aesthetics of failure: post-digital tendencies in contemporary computer music. *Computer Music Journal*, v. 24, n. 4, p. 12–18, 2000.

CLAP. *CLever Audio Plugin specification*. 2024. Disponível em: <https://cleveraudio.org/manual/>.

KLEIDERMACHER DOMINIK; KLEIDERMACHER, M. *Embedded systems security: practical methods for safe and secure software and systems development*. Oxford: Newnes, 2012. ISBN 9780123864785.

LYDA RICHARD A.; HAMROCK, J. K. Using entropy analysis to find encrypted and packed malware. *IEEE Security & Privacy*, v. 5, n. 2, p. 40–45, 2007.

MICROSOFT. *Microsoft PE format specification*. 2024. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>.

PIETREK, M. *Peering inside the PE: a tour of the Win32 Portable Executable file format*. 1994. Disponível em: <https://www.winworldpc.com/blog/peering-inside-the-pe/>.

PIETREK, M. *An in-depth look into the Win32 Portable Executable file format*. 2002. Disponível em: <https://learn.microsoft.com/en-us/archive/blogs/mtpeeb/>.

PIRKLE, W. C. *Designing software synthesizer plug-ins in C++: for RackAFX, VST3, and Audio Units*. 2. ed. New York: Focal Press, 2019. ISBN 9781138566760.

ROADS, C. *Microsound*. Cambridge: MIT Press, 2001. ISBN 9780262681684.

SHANNON, C. E. A mathematical theory of communication. *Bell System Technical Journal*, v. 27, n. 3, p. 379–423, 1948.

SIKORSKI MICHAEL; HONIG, A. *Practical malware analysis: the hands-on guide to dissecting malicious software*. San Francisco: No Starch Press, 2012. ISBN 9781593274073.

SMITH, J. O. *Introduction to digital filters: with audio applications*. [S.l.]: W3K Publishing, 2007. ISBN 9780974884073.

STEINBERG. *VST 3 SDK documentation*. 2024. Disponível em: <https://steinbergmedia.github.io/vst3_dev_portal/>.

ZÖLZER, U. *DAFX: digital audio effects*. 2. ed. Chichester: Wiley, 2011. ISBN 9780470669504.